

Parte 1 — Complejidad Algorítmica (Big O)

1.1 Clasifica la complejidad

- Buscar una palabra en un diccionario físico, abriendo siempre por la mitad de las páginas que quedan.
 - $O(\log n)$. Al dividir la cantidad de páginas a la mitad en cada paso, el espacio de búsqueda se reduce muy rápido
- Revisar, uno por uno, cada carné en una caja de carnés de estudiantes hasta encontrar el que buscas.
 - $O(n)$. En el peor caso se debe revisar todos los carnés uno tras otro
- Sacar la primera carta de un mazo ya barajado.
 - $O(1)$. Es algo directo al primer elemento, no importa cuántas cartas tenga el mazo
- Comparar cada estudiante de un salón con cada uno de los demás estudiantes, para ver si algún par cumple años el mismo día.
 - $O(n^2)$. Comparas todos los estudiantes contra todos los demás, haciendo n por n
- Consultar la hora en tu reloj
 - $O(1)$. Es una sola acción inmediata que no depende de nada más

1.2 De la vida real al análisis

- Lavar platos a mano, si la entrada (n) se hace 10 veces más grande en lugar de 10 platos hay 100 platos tardarás 10 veces más tiempo en lavarlos todos

1.3 ¿Cuál escala mejor?

- Aunque para 5 elementos la Forma A sea rápida la complejidad $O(n^2)$ crece demasiado rápido al aumentar los datos, por ejemplo pasar de 5 a solo 50 elementos la forma A pasa de 25 a 2,500 pasos, mientras que la forma B solo sube a 280

1.4 Verdadero o falso

a. Falso. Big O mide el crecimiento, no los segundos. Un $O(1)$ muy pesado puede tardar más segundos que un $O(n)$ pequeño.

b. Verdadero. Big O mide la tasa de crecimiento de las operaciones, no el tiempo exacto.

c. Falso. Acceder por índice como arreglo[5] es $O(1)$, porque la computadora calcula directamente la ubicación en memoria sin recorrer los elementos anteriores.

Parte 2 — Arreglos 1D

2.1 Diseña el arreglo

- Dibuja el arreglo con sus 6 casillas, mostrando el índice de cada una (recuerda: empieza en 0).

[0]	[1]	[2]	[3]	[4]	[5]
3.5	4.2	2.8	5.0	3.9	4.5

- ¿Qué valor y qué índice tiene la tercera nota que ingresaste?
 - La tercera nota ingresada es 2.8 y está en el índice 2.
- ¿Cuál es la instrucción (en pseudocódigo) para acceder directamente a la nota del último estudiante sin recorrer el arreglo?
 - `notas[5]`

2.2 Direcciones de memoria

Supón que el arreglo `notas` del punto anterior se guarda en memoria RAM empezando en la dirección `0x2000`, y cada valor ocupa 4 bytes.

- Calcula la dirección de memoria de `notas[0]`, `notas[3]` y `notas[5]`, usando la fórmula:
 $\text{dirección} = \text{base} + (\text{índice} \times \text{tamaño})$
 - $\text{notas}[0] = 0x2000 + (0 \times 4) = 0x2000$
 - $\text{notas}[3] = 0x2000 + (3 \times 4) = 0x2000 + 12\text{bytes} = 0x200C$
 - $\text{notas}[5] = 0x2000 + (5 \times 4) = 0x2000 + 20\text{bytes} = 0x2014$
- Explica en tus palabras por qué acceder a `notas[3]` es igual de rápido que acceder a `notas[0]` — no importa cuál pidas, es $O(1)$.
 - Acceder a `notas[3]` es $O(1)$ igual que a `notas[0]` porque la computadora no recorre casilla por casilla solo se calcula con una multiplicación y una suma tardando siempre lo mismo

2.3 Búsqueda lineal vs. binaria

Tienes el arreglo ordenado `edades = [15, 18, 20, 23, 27, 31, 35, 40]`.

- Simula paso a paso una búsqueda lineal para encontrar el valor 31. ¿Cuántas comparaciones hiciste?
 - 0 (15) -> No
 - 1 (18) -> No
 - 2 (20) -> No
 - 3 (23) -> No
 - 4 (27) -> No

5 (31) -> Si
Comparaciones: 6

- Simula paso a paso una búsqueda binaria para encontrar el mismo valor 31.
¿Cuántas comparaciones hiciste?
 - Mitad del arreglo (índice 3 valor 23) No es, 31 es mayor, buscamos a la derecha, mitad de la derecha (índice 5, valor 31) Si es.
Comparaciones: 2
- ¿Por qué la búsqueda binaria sólo funciona si el arreglo ya está ordenado?
 - Se necesita saber si el valor buscado está a la izquierda o a la derecha del centro. Si los números estuvieran desordenados descartar una mitad podría eliminar el número que se busca.
- Si el arreglo tuviera 1 millón de elementos, ¿cuál de las dos búsquedas seguirías usando? Justifica con lo visto sobre $O(n)$ vs. $O(\log n)$.
 - Usaría la binaria. La lineal tardaría hasta 1,000,000 de pasos, mientras que la binaria lo resuelve en unos 20 pasos

2.4 Insertar un elemento

Tienes edades = [15, 18, 20, 23, 27] (5 casillas, sin espacio libre).

- ¿Qué tan costoso es insertar un nuevo valor al final, si hay espacio disponible? ¿Y si no hay espacio y hay que crear un arreglo más grande?
 - Con espacio solo se coloca en la siguiente posición disponible : $O(1)$. Si no hay espacio, toca crear un arreglo más grande y copiar todo: $O(n)$.
- ¿Qué tan costoso es insertar el valor 21 en la mitad (para que quede ordenado)?
Explica qué hay que hacer con los demás elementos.
 - $O(n)$, requiere mover todos los números mayores una casilla hacia la derecha para hacerle espacio.
- Compara ambos casos usando la notación Big O.
 - Insertar al final con espacio: $O(1)$.
 - Sin espacio o en la mitad: $O(n)$.

Parte 3 — Arreglos 2D

3.1 Diseña la matriz

Vas a guardar la disposición de un salón de clase de 3 filas x 4 columnas, donde cada casilla indica si el puesto está ocupado (1) o libre (0).

- Dibuja la matriz salon con sus 3 filas y 4 columnas, con los índices [fila][columna] de cada casilla.

	Columna 0	Columna 1	Columna 2	Columna 3
--	-----------	-----------	-----------	-----------

Fila 0	salon[0][0]	salon[0][1]	salon[0][2]	salon[0][3]
Fila 1	salon[1][0]	salon[1][1]	salon[1][2]	salon[1][3]
Fila 2	salon[2][0]	salon[2][1]	salon[2][2]	salon[2][3]

- Escribe (en pseudocódigo) cómo accedes al puesto de la fila 2, columna 3.
 - salon[2][3]
- ¿Cuántas casillas tiene en total esta matriz? ¿Cómo lo calculas en general para filas × columnas?
 - 12 casillas (se calcula multiplicando filas × columnas).

3.2 De 2D a memoria (row-major)

La memoria RAM es una sola fila continua de casillas — no existen "filas y columnas" físicamente en la RAM. Por eso una matriz se guarda "aplanada", fila por fila.

Supón que salon (3 filas x 4 columnas) se guarda empezando en la dirección 0x1000, con cada valor ocupando 4 bytes, en orden por filas (row-major).

- Dibuja cómo quedarían las 12 casillas de salon en una sola fila de memoria (aplanadas).

[0][0]	[0][1]	[0][2]	[0][3]	[1][0]	[1][1]	[1][2]	[1][3]	[2][0]	[2][1]	[2][2]	[2][3]
--------	--------	--------	--------	--------	--------	--------	--------	--------	--------	--------	--------

- Calcula la dirección de memoria de la casilla [1][2], usando la fórmula:
 $\text{dirección} = \text{base} + ((\text{fila} \times \text{número_de_columnas} + \text{columna}) \times \text{tamaño})$
 - Casilla [1][2]: $0x1000 + ((1 \times 4 + 2) \times 4) = 0x1000 + 24 = 0x1018$
- Calcula también la dirección de [0][0] y de [2][3] (la última casilla).
 - Casilla [0][0]: 0x1000
 - Casilla [2][3]: $0x1000 + ((2 \times 4 + 3) \times 4) = 0x1000 + 44 = 0x102C$

3.3 Recorrido con ciclos anidados

Quieres contar cuántos puestos están ocupados (1) en toda la matriz salon.

- Describe en pseudocódigo el recorrido usando dos ciclos anidados (uno para filas, otro para columnas).
 - ocupados = 0
 - Para f desde 0 hasta filas - 1 Hacer
 - Para c desde 0 hasta columnas - 1 Hacer
 - Si salon[f][c] == 1 Entonces
 - ocupados = ocupados + 1
 - FinSi
 - FinPara
 - FinPara
- Si la matriz tiene f filas y c columnas, ¿cuál es la complejidad de este recorrido en notación Big O? Explica por qué.
 - O(N) respecto al número total de casillas, porque hay que revisar cada casilla una vez.

- ¿Qué pasaría con el tiempo de ejecución si f y c se duplican ambos a la vez?
 - Si f y c se duplican el tiempo se cuadruplica ($2f \times 2c = 4fc$) porque la cantidad total de casillas se multiplica por 4.

3.4 Conectando todo

En un párrafo corto (5-8 líneas), explica con tus propias palabras la relación entre estos tres conceptos vistos en las últimas tres clases: Big O, arreglos 1D y arreglos 2D. Pista: piensa en por qué entender la complejidad te ayuda a decidir cuándo usar un arreglo 1D, cuándo una matriz, y cuándo buscar de forma lineal o binaria.

- Big O para saber qué tan rápido trabaja un programa cuando tiene muchos datos. Los arreglos 1D para guardar datos en una sola fila, mientras que los 2D los organizan en filas y columnas, dependiendo de cómo estén organizados los datos podemos buscar de diferentes formas. La lineal revisa los datos uno por uno, mientras que la binaria es más rápida si los datos están ordenados por eso Big O nos ayuda a elegir la mejor forma de guardar y buscar la información